



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Universidad Autónoma de Nuevo León

Facultad de Ingeniería Mecánica y Eléctrica

Lenguajes de Programación y Lab

Actividad Fundamental 8: Reporte de Programas Imperativos

Unidad 4

Semestre: 3ro

Equipo: 04

Grupo: 025

Salón: 3109

Periodo: Enero – Junio

Docente: Ing. Selene Guadalupe Pinal Torres

Hora: M4-M6

Nombre	Matricula	Carrera
Ruedas Vazquez Jordan Emanuel	2103677	ITS
Enríquez Nungaray Héctor Miguel	2042781	IAS
Hernández Silva Carlos Yahir	1975208	ITS
Heredia López Betsy Aracely	2054413	IAS

Martes 06 de Mayo del 2025

Monterrey, Nuevo León

Contenido

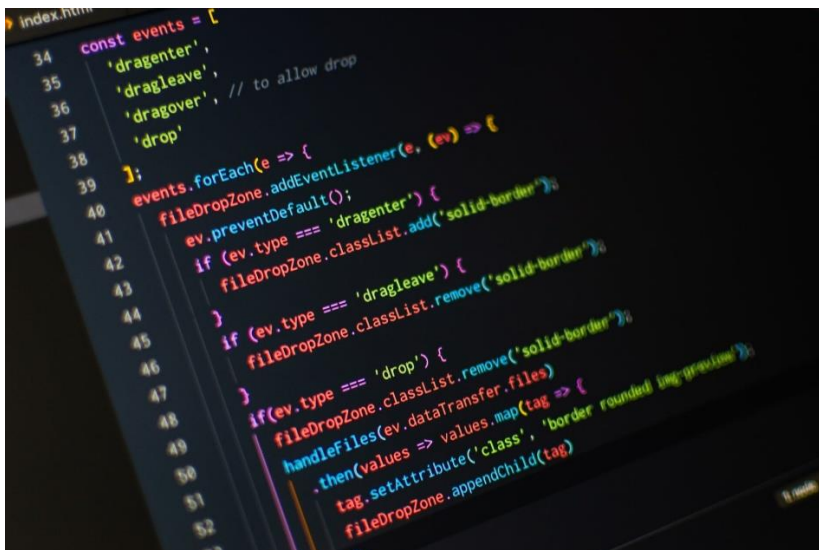
Introducción	2
Definición de lenguaje funcional	3
Características	4
Ventajas y Desventajas	5
Aplicaciones Comunes.....	6
Ejemplos:	7
Conclusión General	12
Conclusiones Individuales.....	13
Referencias Bibliográficas	14

Introducción

La programación imperativa es un paradigma de desarrollo de software que se centra en emitir comandos secuenciales para modificar explícitamente el estado interno de un programa a través de instrucciones como asignaciones, estructuras de control (condicionales y bucles) y llamadas a funciones. A diferencia de los enfoques declarativos, que describen el resultado esperado sin detallar los pasos, la programación imperativa exige al programador especificar cómo se logra cada operación, de manera similar a un conjunto de órdenes o una receta paso a paso. Este estilo de programación se remonta a los primeros lenguajes de bajo nivel (ensamblador y código máquina), donde cada instrucción correspondía directamente a una operación en el hardware, un modelo que perdura en la mayoría de los lenguajes modernos como C, Pascal y Python.

Desde el surgimiento de FORTRAN en 1957, el primer lenguaje de alto nivel que abstraigo parcialmente la complejidad del código máquina, hasta la adopción generalizada de lenguajes como C y Pascal en las décadas de 1970 y 1980, la programación imperativa ha evolucionado incorporando conceptos de estructuración y modularidad. La introducción de bloques de código y subrutinas permitió organizar programas complejos en unidades más manejables, dando origen al paradigma procedural, una forma especializada de programación imperativa. Posteriormente, la aparición de lenguajes orientados a objetos mantuvo la esencia imperativa — control detallado del flujo de ejecución y del estado— al tiempo que añadía estructuras para modelar datos y comportamientos en términos de objetos y clases.

Una de las características distintivas de este paradigma es el uso de variables mutables, que posibilitan cambios de estado a lo largo de la ejecución y brindan un control fino sobre el comportamiento del programa. Las sentencias de asignación actualizan valores en memoria, mientras que los bucles (for, while) permiten repetir operaciones hasta cumplir una condición, y las estructuras condicionales (if, switch) dirigen el flujo de acuerdo con evaluaciones booleanas. Aunque esta exposición detallada del flujo es poderosa y flexible, también conlleva la responsabilidad de gestionar correctamente el estado para evitar errores sutiles como condiciones de carrera o variables no inicializadas.



Definición de lenguaje funcional

La programación imperativa es uno de los paradigmas más antiguos y ampliamente utilizados, basado en la ejecución secuencial de instrucciones que modifican el estado del sistema. En este enfoque, el programador define explícitamente cómo debe lograrse un objetivo, especificando cada paso del proceso. A diferencia de la programación declarativa, que se enfoca en el resultado final sin detallar el procedimiento, la programación imperativa requiere una gestión precisa del flujo de ejecución.

Entre sus principios fundamentales se encuentran:

- **Secuencia de instrucciones:** El orden en que se escriben las instrucciones es crucial, ya que determina el resultado del programa.
- **Estado mutable:** Se hace uso intensivo de variables y estructuras de datos que pueden cambiar durante la ejecución.
- **Estructuras de control:** Bucles, condicionales y funciones son esenciales para dirigir el flujo del programa y tomar decisiones dinámicas.

Este paradigma ofrece un alto nivel de control sobre los recursos del sistema, lo que lo hace compatible con lenguajes como C, Java, Python y muchos otros.

Los lenguajes de programación imperativa más conocidos son:

- ❖ Java
- ❖ Pascal
- ❖ C
- ❖ C++
- ❖ BASIC
- ❖ COBOL
- ❖ Python
- ❖ Ruby

[illegible]

Características

La programación imperativa se caracteriza por enfocarse en la descripción detallada de **cómo** se deben realizar las tareas para alcanzar un resultado específico. En este paradigma, el programador establece paso a paso las instrucciones que debe seguir la computadora, modificando el estado del sistema mediante asignaciones, estructuras de control como condicionales (if, else) y ciclos (for, while). A diferencia de otros paradigmas más declarativos, donde se describe **qué** se desea obtener, la programación imperativa exige una especificación minuciosa del procedimiento, lo que implica una mayor cercanía al funcionamiento interno del hardware. Esta forma de programar se asemeja al pensamiento humano secuencial y es especialmente efectiva para resolver problemas donde es necesario un control riguroso del flujo de ejecución.

Otra característica fundamental es la gestión del **estado del programa** a través del uso de variables y estructuras de memoria. A medida que se ejecutan las instrucciones, el estado del programa cambia en función de los valores asignados a las variables, lo cual permite realizar operaciones complejas mediante una lógica estructurada y fácilmente trazable. Además, este paradigma promueve la programación modular mediante funciones o procedimientos, lo que facilita la organización del código, su mantenimiento y reutilización. Lenguajes como C, Java y Python (en su forma imperativa) son ejemplos populares que siguen este enfoque, y han sido fundamentales en el desarrollo de software desde los sistemas operativos hasta aplicaciones comerciales. La claridad en el flujo de ejecución y la eficiencia en el uso de recursos hacen de la programación imperativa una herramienta poderosa y versátil dentro del campo de la ingeniería de software.



Ventajas y Desventajas

Ventajas

La programación imperativa destaca por ofrecer un control detallado del flujo de ejecución que resulta muy valioso en escenarios donde se requieren optimizaciones de bajo nivel. Al escribir paso a paso las instrucciones que el computador debe seguir, el desarrollador puede afinar cada operación, controlar explícitamente el uso de memoria y ajustar el rendimiento en loops críticos o secciones sensibles al tiempo de respuesta. Este nivel de precisión facilita la identificación y corrección de cuellos de botella, así como la optimización de recursos en aplicaciones de alto rendimiento, sistemas embebidos o videojuegos, donde la eficiencia es esencial.

Otra fortaleza es su intuitividad para los principiantes, ya que el modelo mental de “hacer A, luego B” se asemeja al modo en que las personas construyen procedimientos o recetas en la vida cotidiana. Esta cercanía con la forma de razonar humana simplifica el aprendizaje inicial de la programación, permitiendo que estudiantes y programadores sin experiencia dominen rápidamente conceptos básicos como variables, asignaciones y estructuras de control. Además, el paradigma imperativo cuenta con un amplio respaldo en la industria: lenguajes tan populares como C, Python o Java mantienen un estilo imperativo en su núcleo y disponen de un ecosistema maduro de bibliotecas, herramientas de depuración y documentación, lo cual agiliza el desarrollo y garantiza abundante soporte técnico.

Desventajas

No obstante, a medida que un proyecto crece en tamaño y complejidad, el mantenimiento del código imperativo puede volverse complicado. El manejo explícito del estado global y el encadenamiento de cambios en variables a lo largo de múltiples funciones o módulos genera dependencias difusas que dificultan la lectura y comprensión del programa. Refactorizar o extender funciones requiere identificar cuidadosamente todas las zonas donde esas variables mutan, lo que incrementa el riesgo de introducir fallos al alterar partes interconectadas del sistema.

Además, este paradigma tiende a presentar baja reutilización de código y menor expresividad en determinadas tareas. La tendencia a manipular estructuras de datos paso a paso —por ejemplo, procesar listas o aplicar transformaciones complejas— implica a menudo más líneas de código que en paradigmas declarativos o funcionales, donde operaciones de alto nivel (como filtros, mapeos o recursiones puras) se condensan en una sola expresión. Por último, el uso intensivo de variables mutables y la falta de abstracción de efectos secundarios aumentan el riesgo de errores sutiles, como condiciones de carrera en entornos concurrentes o estados inconsistentes, lo que demanda pruebas y revisiones más exhaustivas para garantizar la fiabilidad del software.

Aplicaciones Comunes

❖ *Sistemas operativos y programación de sistema*

En el desarrollo de **sistemas operativos**, la programación imperativa resulta imprescindible, puesto que lenguajes como C y ensamblador permiten manipular directamente la memoria, los registros del procesador y las interfaces de hardware, garantizando un uso eficiente de los recursos críticos del sistema. Estas capacidades son esenciales para implementar el núcleo del sistema operativo, los controladores de dispositivos y las rutinas de gestión de memoria, donde cada instrucción y cada byte cuentan para mantener la estabilidad y el rendimiento global.

❖ *Aplicaciones embebidas y firmware*

Del mismo modo, en **sistemas embebidos** y desarrollo de firmware para microcontroladores, la cercanía al hardware y la posibilidad de optimizar al máximo el consumo de memoria y energía hacen de la programación imperativa la opción natural. Lenguajes como C o incluso ensamblador se utilizan para programar microcontroladores que rigen desde electrodomésticos hasta vehículos autónomos, donde la predictibilidad y la eficiencia de cada instrucción son críticas para la seguridad y la fiabilidad.

❖ *Videojuegos y motores gráficos*

En el ámbito de los **videojuegos**, los motores gráficos y la lógica del juego dependen en gran medida de rutinas imperativas para gestionar el bucle principal, procesar la entrada del usuario, actualizar el estado del juego y renderizar cada cuadro en tiempo real. Este paradigma permite a los desarrolladores escribir código optimizado que aprovecha al máximo la GPU y la CPU, garantizando una experiencia fluida y de alta fidelidad gráfica.

❖ *Procesamiento de alto rendimiento*

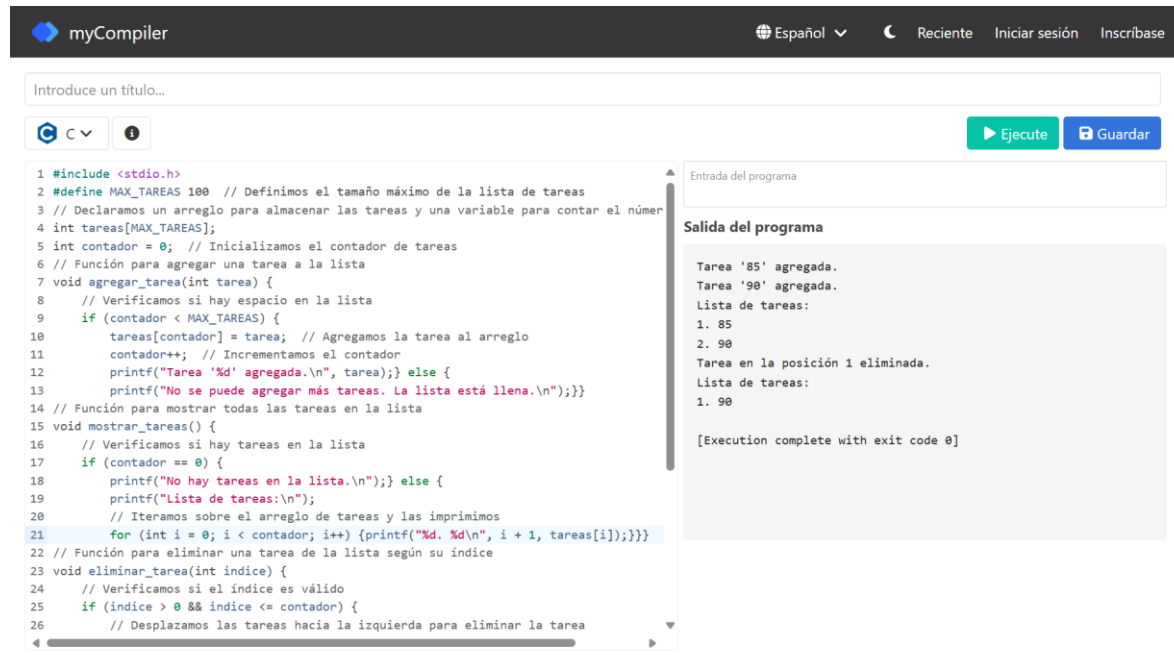
Cuando el **rendimiento es crítico**, como en gráficos por computadora avanzados, simulaciones físicas o procesamiento de señales digitales, la programación imperativa ofrece la capacidad de explotar instrucciones específicas del hardware (SIMD, multihilo, manejo manual de caché) para acelerar cálculos complejos. Gracias al control de bajo nivel sobre la asignación de memoria y la ejecución de bucles, los ingenieros pueden lograr velocidades de ejecución cercanas al límite físico de la máquina.

❖ *Automatización de tareas y scripting*

Finalmente, en la **automatización de tareas** y la creación de **scripts** de mantenimiento, lenguajes imperativos de alto nivel como Bash o Python permiten escribir secuencias de comandos que coordinan herramientas del sistema, realizan copias de seguridad, despliegan aplicaciones o procesan datos de forma repetitiva. La sencillez para expresar flujos de trabajo paso a paso y la disponibilidad de innumerables librerías hacen de estos lenguajes una solución práctica y accesible para administradores de sistemas y desarrolladores.

Ejemplos:

Ruedas Vazquez Jordan Emanuel



```
1 #include <stdio.h>
2 #define MAX_TAREAS 100 // Definimos el tamaño máximo de la lista de tareas
3 // Declaramos un arreglo para almacenar las tareas y una variable para contar el número
4 int tareas[MAX_TAREAS];
5 int contador = 0; // Inicializamos el contador de tareas
6 // Función para agregar una tarea a la lista
7 void agregar_tarea(int tarea) {
8     // Verificamos si hay espacio en la lista
9     if (contador < MAX_TAREAS) {
10         tareas[contador] = tarea; // Agregamos la tarea al arreglo
11         contador++; // Incrementamos el contador
12         printf("Tarea '%d' agregada.\n", tarea);} else {
13             printf("No se puede agregar más tareas. La lista está llena.\n");}
14 // Función para mostrar todas las tareas en la lista
15 void mostrar_tareas() {
16     // Verificamos si hay tareas en la lista
17     if (contador == 0) {
18         printf("No hay tareas en la lista.\n");} else {
19         printf("Lista de tareas:\n");
20         // Iteramos sobre el arreglo de tareas y las imprimimos
21         for (int i = 0; i < contador; i++) {printf("%d. %d\n", i + 1, tareas[i]);}}
22 // Función para eliminar una tarea de la lista según su índice
23 void eliminar_tarea(int indice) {
24     // Verificamos si el índice es válido
25     if (indice > 0 && indice <= contador) {
26         // Desplazamos las tareas hacia la izquierda para eliminar la tarea
```

Entrada del programa

Salida del programa

Tarea '85' agregada.
Tarea '90' agregada.
Lista de tareas:
1. 85
2. 90
Tarea en la posición 1 eliminada.
Lista de tareas:
1. 90
[Execution complete with exit code 0]

Datos de Entrada:

1. agregar_tarea(tarea):

- **tarea (int):** Un número entero que representa la tarea que se desea agregar a la lista.
Ejemplo: 85.

2. eliminar_tarea(indice):

- **indice (int):** Un número entero que representa la posición de la tarea en la lista que se desea eliminar (1 basado). Ejemplo: 1 para eliminar la primera tarea.

Proceso:

1. Agregar Tarea:

- Se llama a la función **agregar_tarea** con un número entero como argumento.
- La función verifica si hay espacio en el arreglo **tareas**.
- Si hay espacio, se agrega la tarea al arreglo y se incrementa el contador de tareas.
- Si no hay espacio, se imprime un mensaje de error.

2. **Mostrar Tareas:**

- Se llama a la función **mostrar_tareas**.
- La función verifica si hay tareas en el arreglo.
- Si hay tareas, se itera sobre el arreglo y se imprimen todas las tareas con su respectivo índice.
- Si no hay tareas, se imprime un mensaje indicando que la lista está vacía.

3. **Eliminar Tarea:**

- Se llama a la función **eliminar_tarea** con un índice como argumento.
- La función verifica si el índice es válido (es decir, si está dentro del rango de la lista).
- Si es válido, se desplazan las tareas restantes hacia la izquierda para llenar el espacio vacío y se decrementa el contador.
- Si el índice no es válido, se imprime un mensaje de error.

Salida:

1. **Agregar Tarea:**

- Un mensaje que confirma que la tarea ha sido agregada. Ejemplo: **"Tarea '85' agregada."**
- Si la lista está llena, se imprime un mensaje de error. Ejemplo: **"No se puede agregar más tareas. La lista está llena."**

2. **Mostrar Tareas:**

- Una lista de las tareas actuales, cada una precedida por su índice. Ejemplo:

RunCopy code

1Lista de tareas:

21. 85

32. 90

3. **Eliminar Tarea:**

- Un mensaje que confirma que la tarea en la posición especificada ha sido eliminada. Ejemplo: **"Tarea en la posición 1 eliminada."**
- Si el índice no es válido, se imprime un mensaje de error. Ejemplo: **"Índice no válido. No se puede eliminar la tarea."**

Enríquez Nungaray Héctor Miguel

Ejemplo

```
main.py
1 # Programa imperativo con interacción del usuario
2
3 # Paso 1: Pedir datos al usuario
4 nombre = input("¿Cuál es tu nombre? ")
5 edad = input("¿Cuántos años tienes? ")
6
7 # Paso 2: Procesar los datos (convertir edad a entero)
8 edad = int(edad)
9
10 # Paso 3: Mostrar mensaje personalizado
11 print("Hola,", nombre + "! En 5 años tendrás", edad + 5, "años.")
12
```

```
¿Cuál es tu nombre? Hector
¿Cuántos años tienes? 20
Hola, Hector ! En 5 años tendrás 25 años.

...Program finished with exit code 0
Press ENTER to exit console.
```

¿Por qué es imperativo?

- Usa instrucciones secuenciales: pedir datos, procesarlos y mostrar resultado.
- Controla el flujo de ejecución con variables.
- Cambia el estado del programa con entradas del usuario.

Heredia López Betsy Aracely

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>Mensaje Personalizado</title>
6 </head>
7 <body>
8   <h1>Mensaje para Betsy</h1>
9   <p id="mensaje"></p>
10
11 <script>
12   // Datos personalizados
13   const nombre = "Betsy Heredia";
14   const edad = 19;
15
16   // Calcular edad futura
17   const edadFutura = edad + 5;
18
19   // Crear mensaje
20   const mensaje = `Hola, ${nombre} ! En 5 años tendrás ${edadFutura} años`;
21
22   // Mostrar mensaje en la página
23   document.getElementById("mensaje").textContent = mensaje;
24 </script>
25 </body>
26 </html>
27
```



Hernández Silva Carlos Yahir

Definimos una lista de números

```
numeros = [10, 20, 30, 40, 50]
```

Inicializamos una variable para la suma

```
suma = 0
```

Recorremos cada número en la lista

```
for numero in numeros:
```

```
    # Sumamos el número actual a la variable suma
```

```
    suma = suma + numero
```

Calculamos el promedio dividiendo la suma por la cantidad de números

```
cantidad = len(numeros)
```

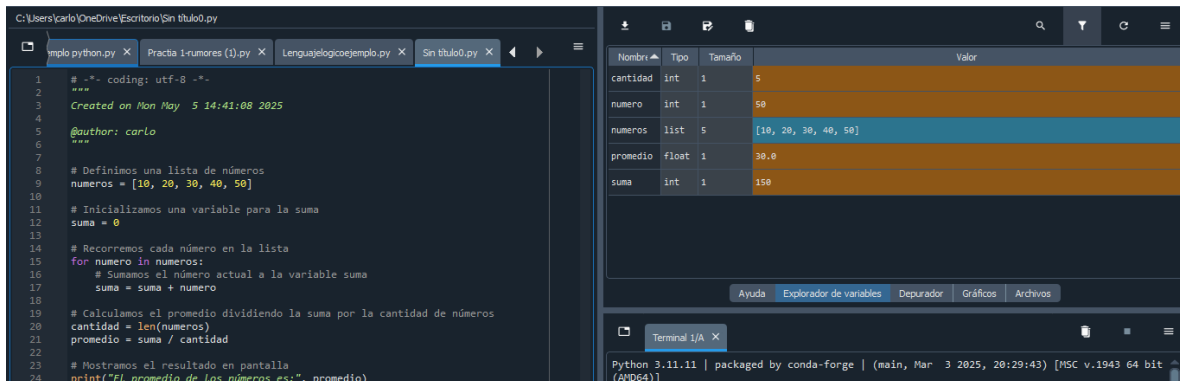
```
promedio = suma / cantidad
```

Mostramos el resultado en pantalla

```
print("El promedio de los números es:", promedio)
```

```
'''
```

Este programa es un ejemplo clásico de programación imperativa en Python que calcula el promedio de una lista de números. Utiliza un enfoque paso a paso donde primero inicializamos variables, luego modificamos su estado a través de un bucle (iterando y acumulando valores) y finalmente realizamos un cálculo con los datos acumulados para obtener el resultado deseado. El programa sigue una secuencia clara de instrucciones que se ejecutan una tras otra, modificando el estado del programa conforme avanza.

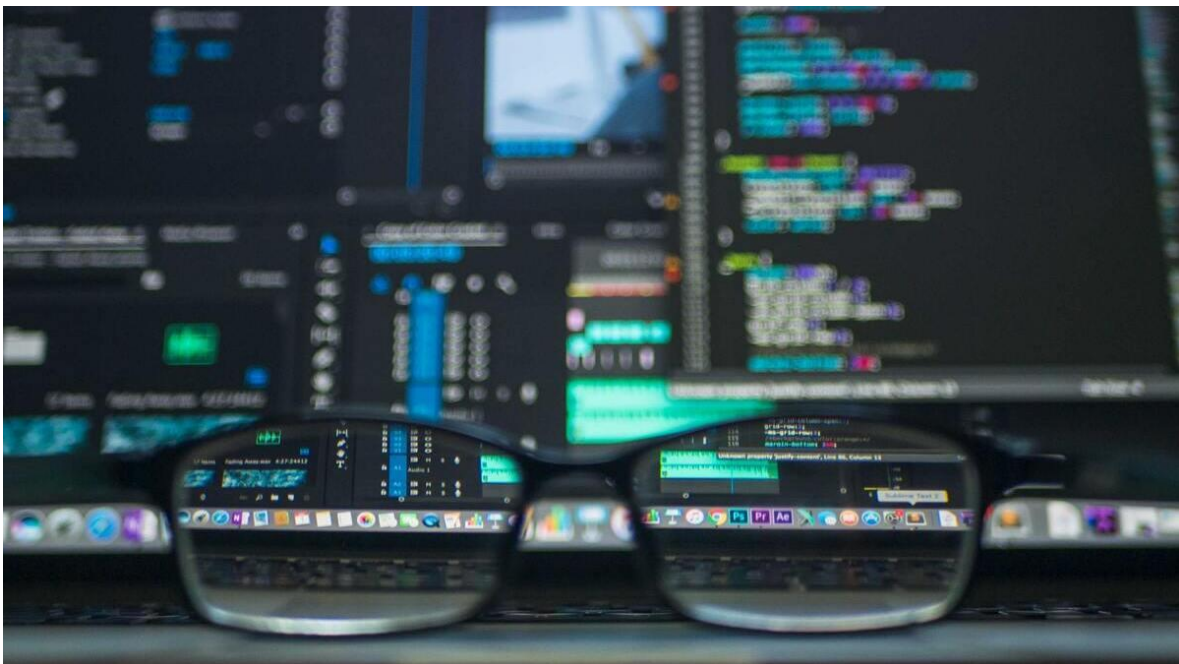


Conclusión General

En definitiva, la programación imperativa constituye la **columna vertebral** de la mayoría de los sistemas de software actuales, gracias a su capacidad para representar algoritmos de manera explícita y controlada. Al obligar al desarrollador a definir cada paso y a administrar directamente el estado de la aplicación, este paradigma facilita un entendimiento profundo del funcionamiento interno de los programas y del hardware sobre el que operan, lo que resulta crucial en entornos de alto rendimiento y sistemas embebidos.

A lo largo de la historia de la informática, desde FORTRAN hasta C y más allá, la programación imperativa ha demostrado su **eficiencia y versatilidad**, permitiendo optimizaciones de bajo nivel que otros paradigmas no siempre facilitan. Sin embargo, su énfasis en los cambios de estado y en la gestión manual del flujo también ha impulsado el desarrollo de paradigmas alternativos —como el orientado a objetos, el funcional o el declarativo— que buscan mitigar problemas de mantenibilidad y errores de estado, ofreciendo distintas abstracciones para describir el comportamiento deseado.

El **aprendizaje** de la programación imperativa sienta unas bases sólidas en el pensamiento algorítmico y en la comprensión de la arquitectura de sistemas, cualidades fundamentales para cualquier desarrollador de software. Dominar este paradigma no solo permite construir aplicaciones eficientes y precisas, sino también entender mejor las fortalezas y limitaciones de otros enfoques, facilitando la elección adecuada según el contexto del proyecto. En síntesis, la programación imperativa continúa siendo esencial por su simplicidad conceptual, su control detallado del flujo de ejecución y su amplia adopción en lenguajes clave de la industria.



Conclusiones Individuales

Ruedas Vazquez Jordan Emanuel

El análisis del paradigma imperativo ha permitido reconocer el valor de una estructura lógica secuencial y controlada en la resolución de problemas computacionales. Comprender cómo se modifica el estado de un programa a través de instrucciones precisas refuerza las bases del pensamiento algorítmico, además de brindar las herramientas necesarias para abordar proyectos reales con un enfoque eficiente y ordenado. Esta experiencia no solo facilita el dominio de lenguajes populares en la industria, sino que también fortalece la capacidad para construir soluciones sistemáticas, comprendiendo en profundidad el comportamiento del código. En consecuencia, se reconoce que la programación imperativa no solo representa una metodología útil, sino también un pilar fundamental en la formación profesional de todo desarrollador de software.

Hernández Silva Carlos Yahir

La programación imperativa es una forma de decirle a la computadora exactamente qué hacer y en qué orden. Al escribir instrucciones paso a paso, el programador tiene un control total sobre cómo se ejecuta el programa y cómo cambian los datos a lo largo del proceso. Este enfoque, aunque puede requerir más trabajo, permite entender claramente cómo funciona todo por dentro. Por eso, sigue siendo una herramienta muy valiosa y ampliamente usada en el desarrollo de software hoy en día.

Referencias Bibliográficas

- Forouzan, B. A., & Gilberg, R. F. (2011). Computer Science: A Structured Programming Approach Using C (3rd ed.). Boston: Cengage Learning.*
- Downey, A. (2016). Think Python: How to Think Like a Computer Scientist (2nd ed.). O'Reilly Media.*
- Sebesta, R. W. (2012). Concepts of Programming Languages (10th ed.). Boston: Pearson Education.*
- Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Englewood Cliffs, NJ: Prentice Hall.*
- Ghezzi, C., & Jazayeri, M. (2003). Programming Language Concepts (3rd ed.). Wiley.*
- Dongee. (2024, 15 de septiembre). Programación imperativa: Ventajas y desventajas [Tutorial]. Recuperado el 5 de mayo de 2025, de <https://www.dongee.com/tutoriales/programacion-imperativa-ventajas-y-desventajas/>*
- IONOS. (2021, 21 de mayo). Programación imperativa: Resumen del paradigma de programación más antiguo [Artículo]. Recuperado el 5 de mayo de 2025, de <https://www.ionos.mx/digitalguide/paginas-web/desarrollo-web/programacion-imperativa/>*
- Quintanar Pérez, D. (2024, 30 de enero). Programación imperativa: Un enfoque paso a paso para resolver problemas de software [Entrada de LinkedIn]. Recuperado el 5 de mayo de 2025, de <https://es.linkedin.com/pulse/programaci%C3%B3n-imperativa-un-enfoque-paso-para-resolver-david-gdrte>*